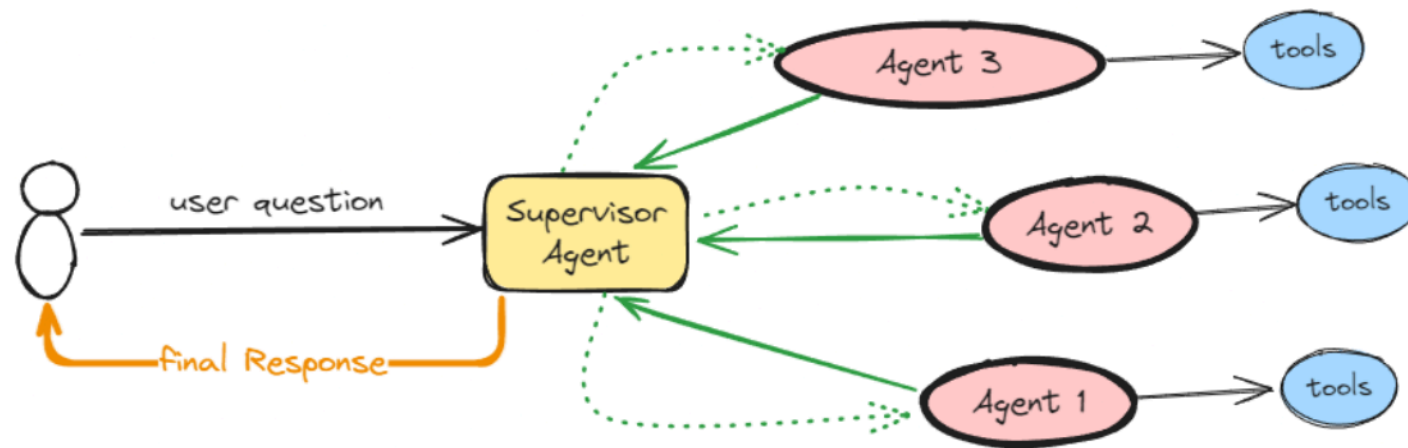




LangGraph

*A framework for building **stateful, multi-step LLM workflows***

LangGraph



LangGraph is an open-source **agent orchestration framework** (built on top of LangChain)



LangGraph lets you control *how* LLM agents think, decide, loop, branch, collaborate, and recover, instead of running a simple linear prompt chain.



At its core, LangGraph combines LLMs with graph-based architecture



By treating workflows as interconnected nodes and edges, LangGraph offers a developer-friendly way to design advanced AI systems like multi-agent system.

Why LangGraph was created

Traditional LLM pipelines (LangChain chains):

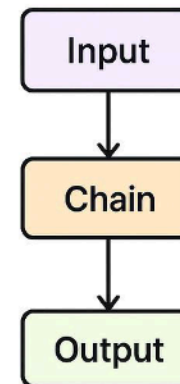
- Linear (Step A → B → C)
- Hard to handle loops, retries, conditional paths
- Poor control over long-running agent workflows
- Difficult to manage multiple agents collaborating

LangGraph solves this by:

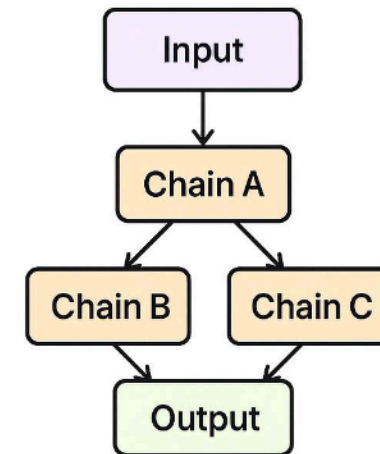
- Representing workflows as a **graph of nodes**
- Maintaining a **shared, persistent state**
- Allowing **conditional routing, loops, human-in-the-loop**
- Supporting **multi-agent collaboration**

LangChain vs. LangGraph Pipelines

LangChain



LangGraph



Core Concepts of LangGraph

Nodes

- Python functions or agent calls
- Function:
 - Call LLMs
 - Call tools
 - Update shared state

State

- Central object passed across nodes which acts like **memory + context**
- Enables:
 - Long conversations, Multi-step reasoning, Decision tracking

Checkpointing

- Save and resume execution
- Useful for:
 - Failures
 - Human approvals
 - Long-running agents

Edges (Control Flow)

- Static edges → fixed flow
- Conditional edges → decision-based routing
- Loops → retries, reflection, validation

Graph

- A **directed graph** of execution
- Nodes = tasks / agents / tools
- Edges = transitions (can be conditional)

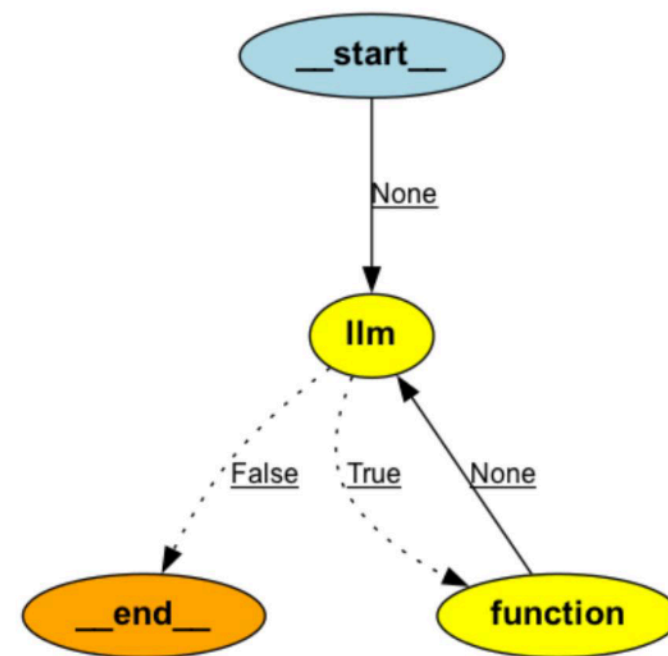
When should you use LangGraph?

Use LangGraph if you need:

- Multi-step reasoning with decisions
- Agents that retry, reflect, or self-correct
- Multiple agents collaborating
- Human-in-the-loop approvals
- Stateful conversations beyond simple chat

Don't use LangGraph if:

- Single prompt → single response
- Simple RAG pipeline
- Stateless chatbot



Use LangGraph when your LLM system is NOT linear

Real-World Applications

1. Multi-Agent AI Systems

- **Planner Agent** → breaks task
- **Research Agent** → gathers data
- **Validator Agent** → checks output
- **Finalizer Agent** → produces response

2. Autonomous Workflow Automation

Flow:

- Detect issue
- Analyze logs
- Propose fix
- Get human approval
- Execute action

3. Human-in-the-Loop AI

- AI drafts a policy
- Sends for human approval
- On rejection → revise automatically
- On approval → publish

4. AI Coding / DevOps Agents

- Read repo
- Identify issue
- Generate fix
- Run tests
- Retry if failed
- Open PR

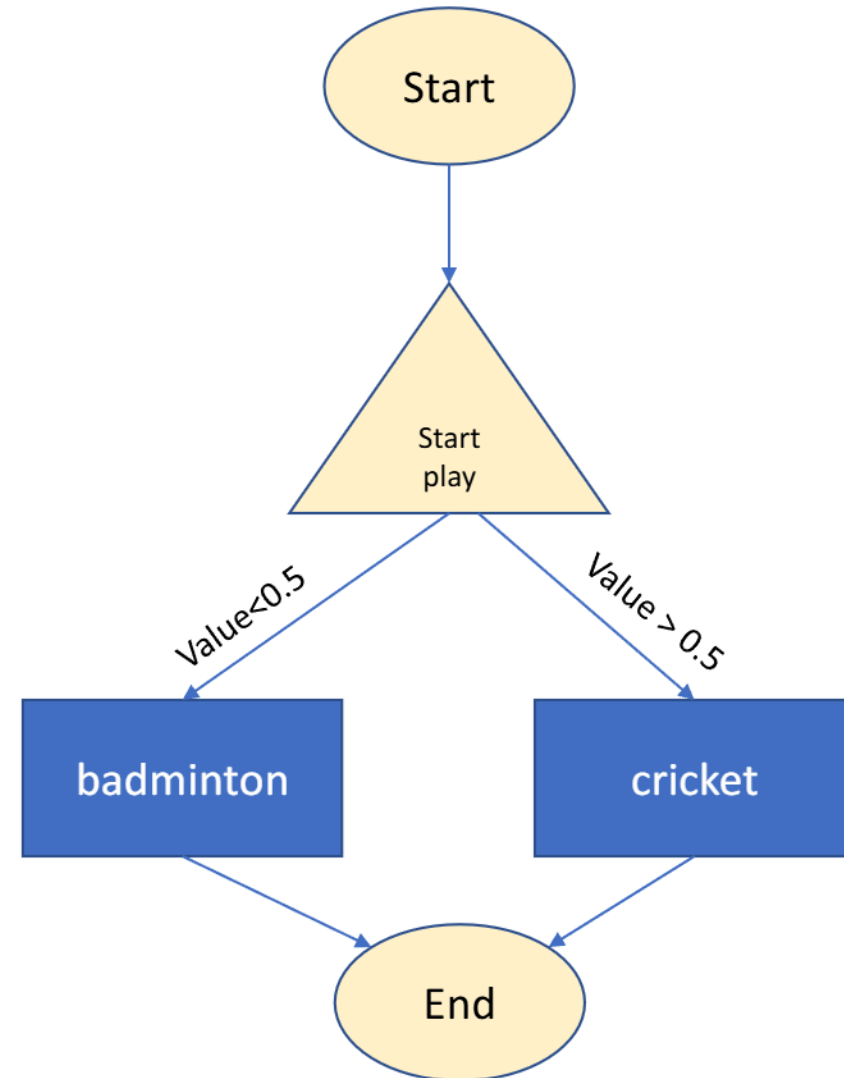


*LangChain tells the LLM what to do.
LangGraph controls how it thinks and
decides.*

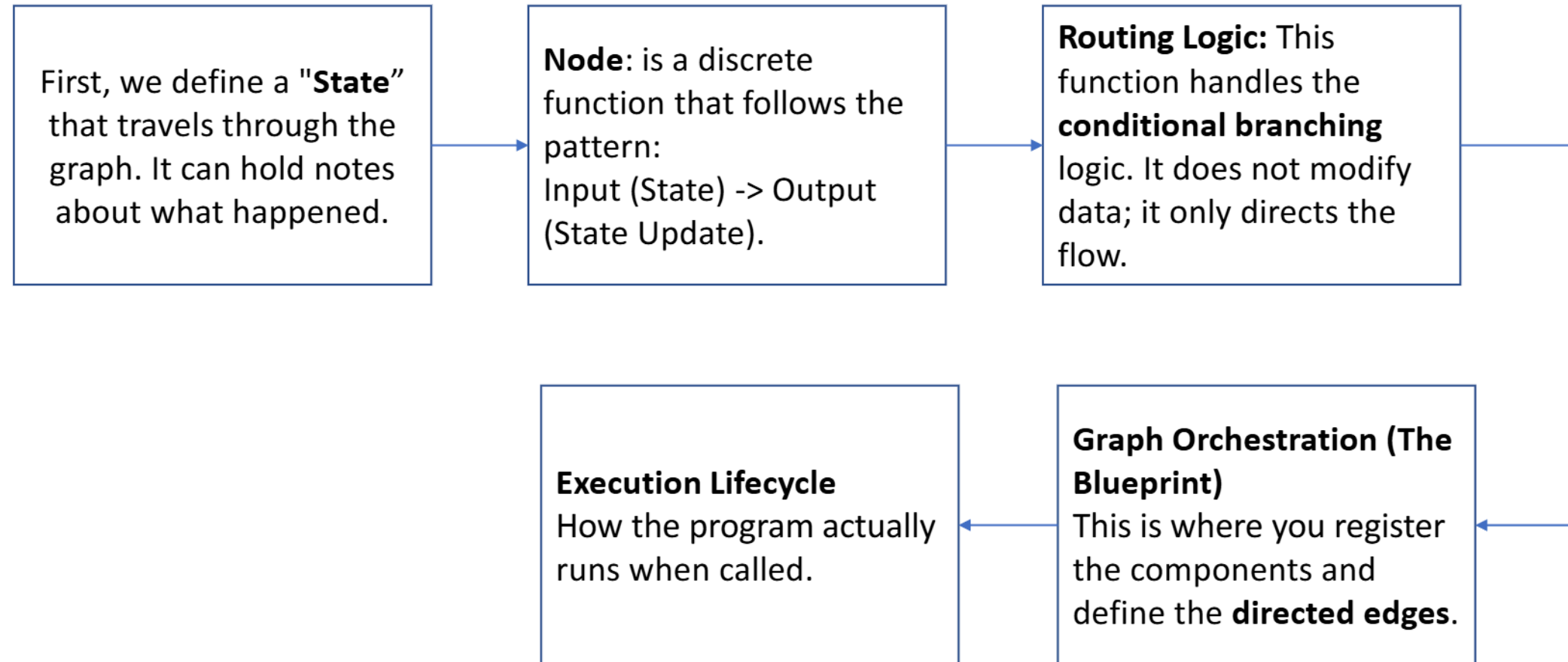


Lets build a simple Graph using state, nodes, edges, conditions

- The goal is to build a **controlled workflow** (a "graph")
- It starts a game session and then uses a "coin flip" (random logic) to decide whether the player will play Cricket or Badminton.
- It tracks the history of these actions in a shared memory object called the "State."



Pseudocode





Now lets build using OpenAI, Langchain

Obj: Build a simple LangGraph workflow that routes a user query to either a calculator or a text responder based on the input content using shared state and conditional graph execution.



THANK YOU
